RA26-Challenge 6: Solución Entrenamiento

DMA, scroll, mapas, window, niveles, IA, coordenadas y animación

ⓘ Cómo usar esta solución
Intenta resolver el ejercicio antes de mirar. Compara con tu solución: no tiene que ser idéntica. Debe cumplir el contrato y debes entender cada bloque.
Los comentarios con `\ | /` agrupan las instrucciones en bloques lógicos. Cada ejercicio indica su bloque de la guía de aprendizaje.

Ejercicio 1: DMARoutine y dma_init: copiar sprites con DMA (corregir el código)

Los errores

Tipo	Dónde	Corrección
N	`ld a, DMA_WAIT_LOOP`	`DMA_WAIT_LOOP` no existe (lo dice el ensamblador): la constante es `DMA_WAIT_LOOPS`.
E	`jr z, .wait`	Tras el primer `dec`, `A` vale 39, que no es 0: `jr z` no salta y la espera acaba con la copia a medias. Hay que repetir mientras `A` no llegue a 0: `jr nz, .wait`.
F	falta `ret`	Sin `ret`, la CPU sigue ejecutando lo que haya en HRAM detrás de la rutina. `dma_run` la ejecuta con `jp`, así que este `ret` vuelve a quien llamó a `dma_run`.
E	`ld hl, hDMARoutine` y `ld de, DMARoutine`	`memcpy8` copia de `HL` (origen) a `DE` (destino). El origen es la rutina en la ROM, `DMARoutine`, y el destino, `hDMARoutine` en HRAM: están intercambiados.

ⓘ Pasos de la solución

1. Escribe a un lado lo que debe hacer `DMARoutine`: escribir `HIGH(shadow_oam)` en `rDMA`, esperar `DMA_WAIT_LOOPS` vueltas y volver.
2. Ensambla: `DMA_WAIT_LOOP` no existe. La constante de `dma.asm` es `DMA_WAIT_LOOPS`.
3. Cuenta las vueltas del bucle con su condición: con `jr z` sale en la primera vuelta. Tiene que repetir mientras `A` no sea 0: `jr nz`.
4. Mira cómo termina: después del bucle no hay nada. Falta `ret`.
5. `dma_init`, con el contrato de `memcpy8` delante: `HL` = origen (`DMARoutine`, en la ROM), `DE` = destino (`hDMARoutine`, en HRAM), `B` = tamaño. `HL` y `DE` estaban intercambiados.

DMARoutine y dma_init (corregida)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; This is the code that will run from HRAM (see dma_init). Its addresses are absolute
;; (ldh [rDMA]...) and it uses no jumps to the outside, so it works from any place.
DMARoutine:
   ld   a, HIGH(shadow_oam)     ;; \ rDMA = the page of the shadow OAM:
   ldh  [rDMA], a               ;; / the copy starts now
   ld   a, DMA_WAIT_LOOPS       ;; \ Wait for the copy to end,
.wait                           ;; | doing nothing (only HRAM can be read):
   dec  a                       ;; | 40 loops x 4 cycles
   jr   nz, .wait               ;; /
   ret
DMARoutineEnd:

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Copies the DMA routine to HRAM. It is done once, when the console starts.
;; ❌ Modifies A, B, DE, HL
dma_init::
   ld   hl, DMARoutine          ;; \ HL = the routine in the ROM
   ld   de, hDMARoutine         ;; | DE = its place in HRAM
   ld   b, DMARoutineEnd - DMARoutine ;; | B = its size
   jp   memcpy8                 ;; / Copy it
```

Casos de prueba (P1 y P2)

- **P1, dma_init.** dma_init copia los 10 bytes de DMARoutine (3E C1 E0 46 3E 28 3D 20 FD C9) a HRAM. Los otros 6 bytes siguen en EE.
- **P2, la copia.** La OAM queda como la shadow_oam: 00, 01 ... 9F. Se muestran los primeros y los últimos 16 bytes de los 160.

Fíjate en

- `DMA_WAIT_LOOPS` = 40 y cada vuelta cuesta 4 ciclos (`dec` 1 + `jr nz` 3): `40 × 4 = 160` , lo que dura la copia.
- El tamaño lo calcula el ensamblador (`DMARoutineEnd - DMARoutine`): al añadir el `ret` , la rutina pasa a 10 bytes sin tocar `dma_init` .

Errores típicos

Nos pasa a todos:

- Arreglar el bucle y olvidar el `ret` : el juego se cuelga en el primer frame.
- Ver que algo falla en `dma_init` y cambiar solo uno de los dos registros.

Ejercicio 2: camera_axis y sys_render_scroll: la cámara y el scroll

Pasos de la solución

1. `camera_axis` : `sub b` calcula `A` = posición − offset. Si la posición es menor que el offset, la resta da la vuelta y activa el acarreo.

2. Sin acarreo, `jr nc` sigue en `.not_negative`. Con acarreo, `xor a` (`A` = 0) y `ret`: la cámara no baja de 0.
3. `cp c` calcula `A - C` sin guardarlo: acarreo si `A < C`. Entonces `ret c`: `A` es la cámara.
4. Si no, `ld a, c` y `ret`: la cámara se queda en el máximo.
5. `sys_render_scroll`: `call sys_camera_get` deja `B` = cámara Y y `C` = cámara X.
6. Escribe cada registro pasando por `A`: `ld a, c` y `ldh [rSCX], a`; `ld a, b` y `ldh [rSCY], a`. `ret`.

camera_axis y sys_render_scroll

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; The camera for ONE axis: position - offset, not below 0 and not above the maximum.
;; 📥 A = position of the entity in that axis, B = offset, C = maximum of the camera
;; 🡸 A = the camera in that axis
;; ❌ Modifies A
camera_axis::
   sub  b                       ;; \ A = position - offset
   jr   nc, .not_negative       ;; / No carry: it did not go below 0

   xor  a                       ;; \ Below 0 (it wrapped around): the camera is 0
   ret                          ;; /

.not_negative
   cp   c                       ;; \ Is A below the maximum?
   ret  c                       ;; / Yes: it is the camera

   ld   a, c                    ;; \ No: the camera stays at its maximum
   ret                          ;; /

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves the background to the position of the camera: the scroll. The registers
;; rSCX and rSCY say which pixel of the background is at the top left of the screen.
;; ⚠️ Changing them in the middle of a frame tears the picture: do it in VBLANK
;; (or with the screen off). Use sys_camera_get
;; ❌ Modifies A, BC
sys_render_scroll::
   call sys_camera_get          ;; B = camera Y, C = camera X

   ld   a, c                    ;; \ rSCX = camera X
   ldh  [rSCX], a               ;; /
   ld   a, b                    ;; \ rSCY = camera Y
   ldh  [rSCY], a               ;; /

   ret
```

Casos de prueba (P1 y P2)

- **P1, caso normal.** camera_axis: 120 − 72 = 48 (30 en hexadecimal), el valor de A, que se guarda en tst_log. sys_render_scroll: rSCX = cam_x = 48 (30) y rSCY = cam_y = 30 (1E).
- **P2, los límites.** camera_axis(30): 30 − 72 baja de 0 (acarreo) y devuelve 0. camera_axis(250): 250 − 72 = 178, que pasa del máximo, y devuelve 96 (60 en hexadecimal). sys_render_scroll deja rSCX = rSCY = 0.

Fíjate en

- `xor a` es la forma corta de poner `A` a 0.
- Si `A` es igual que `C`, `cp c` no da acarreo y se devuelve `C`: el mismo valor.

⚠ Errores típicos

Nos pasa a todos:

- Intercambiar `B` y `C` al escribir los registros: la Y de la cámara en `rSCX`.
- Devolver el resultado de la resta cuando hay acarreo: la cámara salta al otro extremo del mundo.

Ejercicio 3: map_tile_address y map_is_solid_at: el tile de un punto y si es sólido

ⓘ Pasos de la solución

1. Haz la cuenta de P1 a mano: `Y` = 216, `X` = 32. Fila = 27, columna = 4, offset = `27 × 32 + 4 = 868`.
2. `ld a, d` y `and %11111000`: `A` = `Y` sin sus 3 bits bajos = `fila × 8` (216 = 27 × 8).
3. `ld l, a` y `ld h, 0`: `HL` = `fila × 8`, en 16 bits, porque `fila × 32` llega a 992.
4. Dos `add hl, hl`: `HL` = `fila × 32`.
5. `ld a, e` y tres `srl a`: `A` = `X / 8` = columna.
6. `add l` y `ld l, a`: `HL` = `fila × 32 + columna`. No hay acarreo: solo cambia `L`.
7. El código dado suma `map_current` (con `add` y `adc`) y vuelve.
8. `map_is_solid_at`: `call map_tile_address` deja en `HL` la dirección del tile. `DE` no cambia: `map_tile_address` no lo modifica.
9. `ld a, [hl]` y `cp TILE_FIRST_SOLID`: con acarreo, el tile es menor y está libre (`jr c, .free`).
10. Sólido: `or 1` deja `NZ` (`A` no puede ser 0) y `ret`.
11. `.free`: `xor a` deja `Z` y `ret`.

map_tile_address y map_is_solid_at

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Where is the tile that covers a point of the world?
;; The map is a table of rows of MAP_WIDTH (32) tiles, and a tile is 8 x 8 pixels:
;;      tile row    = Y / 8        tile column = X / 8
;;      address     = map + row * 32 + column
;; 📥 D = Y, E = X (pixels of the world)
;; 🔙 HL = address of its tile in the map
;; ❌ Modifies A, HL
map_tile_address::
   ld   a, d                    ;; \ A = Y without its 3 low bits = row * 8
   and  %11111000               ;; |
   ld   l, a                    ;; | HL = row * 8
   ld   h, 0                    ;; /
   add  hl, hl                  ;; \ HL = row * 8 * 2 * 2 = row * 32
   add  hl, hl                  ;; / (the start of its row in the map)

   ld   a, e                    ;; \ A = X / 8 = column
   srl  a                       ;; |
   srl  a                       ;; |
   srl  a                       ;; /
   add  l                       ;; \ HL += column: row * 32 has its 5 low bits at 0
   ld   l, a                    ;; / and column < 32, so no carry: only L changes

   ld   a, [map_current]        ;; \ HL += map_current: the address in the map
   add  l                       ;; |
   ld   l, a                    ;; |
   ld   a, [map_current + 1]    ;; |
   adc  h                       ;; |
   ld   h, a                    ;; /

   ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Is the point solid? A tile is solid if it is TILE_FIRST_SOLID or higher:
;; the tiles are ordered in the tile block so that one comparison decides it.
;; 📥 D = Y, E = X (pixels of the world)
;; 🔙 NZ, the point is in a solid tile
;;    Z,  it is free
;; ❌ Modifies A, HL. Does NOT modify DE
;; ⚠️ Use map_tile_address
map_is_solid_at::
   call map_tile_address        ;; HL = the tile that covers the point

   ld   a, [hl]                 ;; \ A = the tile
   cp   TILE_FIRST_SOLID        ;; | Is it below the first solid tile?
   jr   c, .free                ;; / Yes: it is free

   or   1                       ;; \ NZ: solid
   ret                          ;; /

.free
   xor  a                       ;; \ Z: free
   ret                          ;; /
```

☰ Casos de prueba (P1 y P2)

- **P1, un punto libre.** map_tile_address: fila = 216 / 8 = 27 y columna = 32 / 8 = 4: 27 × 32 + 4 = 868 (364 en hexadecimal). HL = Level1Map + 364 = 04EC, guardado en tst_log como EC 04. map_is_solid_at: ese tile es un camino (TILE_PATH = 67, 43 en hexadecimal, menor que TILE_FIRST_SOLID = 72): libre, Z = 1. HL vuelve a ser 04EC y DE no cambia.
- **P2, un punto sólido.** map_tile_address: fila = 31 y columna = 31: 31 × 32 + 31 = 1023 (3FF en hexadecimal). HL = Level1Map + 3FF = 0587, el último byte del mapa, guardado en tst_log como 87 05. map_is_solid_at: ese tile es un árbol (TILE_TREE = 72 = TILE_FIRST_SOLID): sólido, Z = 0. HL = 0587 y DE no cambia.

ⓘ Fíjate en

- `fila × 32` tiene sus 5 bits bajos a 0 y la columna es menor que 32: la suma solo rellena esos 5 bits.
- Tras `cp`, un árbol (`TILE_TREE` = `TILE_FIRST_SOLID`) deja `Z` a 1: por eso el caso sólido pone `NZ` con `or 1` en vez de fiarse de los flags de `cp`. P2 es justo un árbol.

⚠ Errores típicos

Nos pasa a todos:

- Duplicar `HL` antes de poner `H` a 0: `H` lleva lo que hubiera.
- Desplazar `X` 2 o 4 veces en lugar de 3.
- Usar `A` para la columna antes de guardar `fila × 8` en `L`.
- En `map_is_solid_at`, `jr nc` en lugar de `jr c`: libre y sólido quedan al revés.

Ejercicio 4: hud_show y hud_draw_hearts: el HUD en la window (completar)

ⓘ Pasos de la solución

1. `hud_show`: `ld a, HUD_WINDOW_X` y `ldh [rWX], a`; `ld a, HUD_WINDOW_Y` y `ldh [rWY], a`. `ret`.
2. `hud_draw_hearts` ya trae `B` = `HEARTS_MAX` y `HL` = la primera celda.
3. En cada vuelta, `ld a, c` y `or a`: `Z` si no quedan corazones llenos por pintar.
4. `ld a, TILE_HEART_EMPTY` no cambia los flags: con `Z`, `jr z, .put`.
5. Si quedan: `ld a, TILE_HEART_FULL` y `dec c`.
6. `.put`: `ld [hl+], a` escribe la celda y avanza.
7. `dec b` y `jr nz, .heart` mientras queden celdas. `ret`.

hud_show y hud_draw_hearts

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Shows the window at the bottom of the screen.
;; ⚠️ The screen must be OFF (or be in VBLANK)
;; ❌ Modifies A
hud_show::
   ld   a, HUD_WINDOW_X         ;; \ The window starts at the left of the screen
   ldh  [rWX], a                ;; /
   ld   a, HUD_WINDOW_Y         ;; \ and at the last row of tiles
   ldh  [rWY], a                ;; /

   ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Draws the hearts of the HUD: HEARTS_MAX cells, the first C full and the rest empty.
;; 📥 C = hearts the hero has
;; ❌ Modifies A, BC, HL
hud_draw_hearts::
   ld   b, HEARTS_MAX           ;; B = hearts to draw
   ld   hl, HUD_HEARTS_CELL     ;; HL = the first cell of the hearts

.heart
   ld   a, c                    ;; \ Does the hero have a heart left to show?
   or   a                       ;; |
   ld   a, TILE_HEART_EMPTY     ;; | A = an empty heart, if it does not
   jr   z, .put                 ;; |
   ld   a, TILE_HEART_FULL      ;; | A = a full heart
   dec  c                       ;; / One heart less to show
.put
   ld   [hl+], a                ;; Write it in the cell

   dec  b                       ;; \ B--: next heart,
   jr   nz, .heart              ;; / while there are cells left

   ret
```

Casos de prueba (P1 y P2)

- **P1, dos corazones.** hud_show: rWY = HUD_WINDOW_Y = 136 (88 en hexadecimal) y rWX = HUD_WINDOW_X = 7. hud_draw_hearts con C = 2: dos corazones llenos (TILE_HEART_FULL = 69, 45 en hexadecimal) y uno vacío (TILE_HEART_EMPTY = 70, 46 en hexadecimal).
- **P2, sin corazones.** Con C = 0, las tres celdas son corazones vacíos (46 en hexadecimal).

Fíjate en

- Comprobar antes y cargar el tile después funciona porque `ld a, n` no toca los flags.

⚠ Errores típicos

Nos pasa a todos:

- Restar a `C` antes de comprobarlo: con `C` = 0 da 255 y pinta todos llenos.
- Escribir con `ld [hl], a` sin avanzar: los tres corazones en la misma celda.

Ejercicio 5: game_enemies_spawn: crear los enemigos de un nivel

ⓘ Pasos de la solución

1. `ld a, [hl+]` : `A` = número de enemigos, y `HL` pasa a la primera posición.
2. `ld [enemies_left], a` .
3. `ld d, h` y `ld e, l` : `DE` recorre la lista, porque `game_spawn_at` no modifica `DE` .
4. `or a` y `ret z` : sin enemigos, termina.
5. En cada vuelta, `push af` guarda cuántos quedan: `game_spawn_at` modifica `A` , `BC` y `HL` .
6. `ld a, [de]` , `inc de` y `ld c, a` : `C` = columna. Igual para `B` = fila.
7. `ld hl, EnemyTemplate` y `call game_spawn_at` .
8. `pop af` , `dec a` y `jr nz, .enemy` mientras queden. `ret` .

game_enemies_spawn

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Creates the enemies of a level. In the data of the level, a byte with how many
;; they are is followed by the position of each one: its column and row, in tiles.
;; It also remembers how many there are (enemies_left).
;; 📥 HL = &level.ENEMIES
;; ❌ Modifies A, BC, DE, HL
;; ⚠️ Use game_spawn_at, with EnemyTemplate
game_enemies_spawn::
   ld   a, [hl+]                ;; \ enemies_left = the number of enemies
   ld   [enemies_left], a       ;; | (HL -> their positions)
   ld   d, h                    ;; | DE = the positions
   ld   e, l                    ;; /
   or   a                       ;; \ No enemies? Nothing else to do
   ret  z                       ;; /

.enemy
   push af                      ;; Save the enemies left to create
   ld   a, [de]                 ;; \ C = the column of the enemy
   inc  de                      ;; |
   ld   c, a                    ;; /
   ld   a, [de]                 ;; \ B = its row
   inc  de                      ;; |
   ld   b, a                    ;; /
   ld   hl, EnemyTemplate       ;; \ Create the enemy in that tile
   call game_spawn_at           ;; / (DE is not modified)
   pop  af                      ;; \ One enemy less to create,
   dec  a                       ;; | while there are enemies left
   jr   nz, .enemy              ;; /

   ret
```

⁝☰ Casos de prueba (P1 y P2)

- **P1, 3 enemigos.** enemies_left = 3. Tres enemigos, en el orden de los datos, con Y = fila × 8 y X = columna × 8. Primero (columna 2, fila 3): Y = 24 (18), X = 16 (10). Segundo (20, 4): Y = 32 (20), X = 160 (A0). Tercero (9, 30): Y = 240 (F0), X = 72 (48). El resto de cada entidad viene de EnemyTemplate.

- **P2, ningún enemigo.** enemies_left = 0 y no se crea ninguna entidad.

ⓘ Fíjate en

- Sin la comprobación del 0, el bucle daría 256 vueltas (`dec a` de 0 da 255).

⚠ Errores típicos

Nos pasa a todos:

- Guardar el contador en `B` o en `C` : `game_spawn_at` los sobreescribe.
- Leer la fila y la columna al revés: los datos son columna, fila.

Ejercicio 6: sys_ai_one: decidir cada cierto tiempo

ⓘ Pasos de la solución

1. `HL` = `&entidad.THINK` : `ld h, d` , `ld a, e` , `add E_THINK` y `ld l, a` (misma página: sin acarreo).
2. `dec [hl]` y `ret nz` : si `THINK` no llega a 0, no toca decidir.
3. `push hl` , `call rng_next` , `ld c, a` y `pop hl` : `rng_next` modifica `HL` ; el número queda en `C` , que `rng_next` no toca.
4. `and AI_THINK_MASK` , `add AI_THINK_MIN` y `ld [hl], a` : el nuevo `THINK` .
5. `ld a, c` y `jp sys_ai_choose` : elige con el mismo número. `DE` sigue siendo la entidad, y el `ret` de `sys_ai_choose` vuelve a quien llamó.

sys_ai_one

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; AI system, for ONE entity: every so often, it chooses what to do next.
;; Its THINK counts the frames left before it chooses. When it reaches 0:
;;  - THINK = AI_THINK_MIN + a random number (0-31): the time until the next choice.
;;  - It chooses what to do with that same random number (sys_ai_choose).
;; ⚠️ Precondition: the entity has CMP_AI.
;; 📥 DE = entity
;; ❌ Modifies A, BC, HL
sys_ai_one::
   ld   h, d                    ;; \ HL = &entity.THINK: the same page as the entity,
   ld   a, e                    ;; | the offset added to the low byte (no carry)
   add  E_THINK                 ;; |
   ld   l, a                    ;; /
   dec  [hl]                    ;; \ One frame less to think:
   ret  nz                      ;; / it is not time yet

   push hl                      ;; Save &THINK (rng_next changes HL)
   call rng_next                ;; A = a random number
   ld   c, a                    ;; C = the random number
   pop  hl                      ;; HL = &THINK

   and  AI_THINK_MASK           ;; \ THINK = AI_THINK_MIN + (its 5 low bits)
   add  AI_THINK_MIN            ;; |
   ld   [hl], a                 ;; /

   ld   a, c                    ;; A = the random number again
   jp   sys_ai_choose           ;; Choose what to do with it (and return)
```

Casos de prueba (P1 y P2)

- **P1, toca elegir.** THINK pasa de 1 a 0: toca elegir. rng_next consume un número (rng_state pasa a 21 29) y THINK = AI_THINK_MIN + (número AND AI_THINK_MASK) = 33 (21). Con ese mismo número, el bit 7 vale 0 y los bits 6-5 valen 01: la entidad camina hacia DIR_UP (DIR = 01, VY = −1 = FF).
- **P2, todavía no.** THINK pasa de 5 a 4: todavía no toca elegir. No cambia nada más (VY sigue en 1) y no se llama a rng_next: rng_state sigue en A5 5A.

Fíjate en

- `dec [hl]` activa `Z` cuando el valor llega a 0: no hace falta leerlo.

⚠ Errores típicos

Nos pasa a todos:

- Olvidar `push hl` / `pop hl`: el nuevo `THINK` se escribe en otra dirección.
- Elegir con `A` después del `and`: ya no es el número aleatorio entero.

Ejercicio 7: sys_ai_choose: quieta o caminando

ⓘ Pasos de la solución

1. `swap a` intercambia los nibbles: los bits 7-5 pasan a las posiciones 3-1.
2. `rrca` los baja una más: el bit 7 queda en el bit 2 ("quieta") y los bits 6-5, en los bits 1-0 (la dirección).
3. `bit AI_STILL_BIT, a` y `jp nz, sys_physics_stop`.
4. `and DIR_MASK` y `ld b, a` : `B` = dirección. `jp sys_physics_walk`.

sys_ai_choose

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; The AI decides what an entity does, from a random number: half of the times it
;; stands still, and the other half it walks in one of the 4 directions.
;;  - Bit 7 of the number: 1 = stand still (sys_physics_stop).
;;  - Bits 6-5 of the number: the direction to walk (sys_physics_walk, B = direction).
;; 📥 DE = entity, A = a random number
;; ❌ Modifies A, BC, HL
;; ⚠️ sys_physics_stop and sys_physics_walk end with their own ret
sys_ai_choose::
   swap a                       ;; \ The 3 high bits of the number, to the low bits
   rrca                         ;; /
   bit  AI_STILL_BIT, a         ;; \ Stand still this time?
   jp   nz, sys_physics_stop    ;; / Yes: stop (and return)

   and  DIR_MASK                ;; \ B = the direction (DIR_DOWN ... DIR_RIGHT)
   ld   b, a                    ;; /
   jp   sys_physics_walk        ;; Walk that way (and return)
```

⁝☰ Casos de prueba (P1 y P2)

- **P1, se queda quieta.** El bit 7 de A vale 1: la entidad se queda quieta. VY pasa de 1 a 0; VX y DIR no cambian.
- **P2, camina.** El bit 7 vale 0 y los bits 6-5 valen %11 = DIR_RIGHT (3): camina a la derecha. DIR = 03 y VX = 1 (WALK_SPEED).

ⓘ Fíjate en

- P2: `A` = %01110000; tras `swap a`, %00000111; tras `rrca`, %10000011. El bit 2 vale 0 (camina) y `and DIR_MASK` da 3 = `DIR_RIGHT`.
- Hay otros caminos válidos (por ejemplo, `bit 7, a` antes de desplazar): lo que cuenta es el resultado.

⚠ Errores típicos

Nos pasa a todos:

- Pasar a `sys_physics_walk` los bits 6-5 sin bajarlos: `B` valdría 0, 32, 64 o 96.
- Usar `call` sin `ret` detrás: la rutina seguiría ejecutando lo siguiente.

Ejercicio 8: world_to_oam: del mundo a la OAM

ⓘ Pasos de la solución

1. `call sys_camera_get`: `B` = cámara Y y `C` = cámara X. `DE` y `HL` no cambian.
2. `ld a, d`, `sub b`, `add OAM_Y_OFFSET` y `ld b, a`: `B` = Y en la OAM.
3. `ld a, e`, `sub c`, `add OAM_X_OFFSET` y `ld c, a`: `C` = X en la OAM.
4. `ret`.

world_to_oam

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Changes a position from WORLD coordinates to OAM coordinates:
;;    OAM = world - camera + offset   (offset: OAM_Y_OFFSET for Y, OAM_X_OFFSET for X)
;; All in 8 bits: if the position is outside the screen, the result wraps around to
;; values that the console does not draw (Y >= 160 or X >= 168).
;; 📥 D = world Y, E = world X
;; 🡸 B = OAM Y, C = OAM X
;; ❌ Modifies A, BC. Does NOT modify DE or HL
;; ⚠️ Use sys_camera_get
world_to_oam::
   call sys_camera_get          ;; B = camera Y, C = camera X (DE and HL do not change)

   ld   a, d                    ;; \ B = OAM Y = world Y - camera Y + OAM_Y_OFFSET
   sub  b                       ;; |
   add  OAM_Y_OFFSET            ;; |
   ld   b, a                    ;; /

   ld   a, e                    ;; \ C = OAM X = world X - camera X + OAM_X_OFFSET
   sub  c                       ;; |
   add  OAM_X_OFFSET            ;; |
   ld   c, a                    ;; /

   ret
```

⁝≡ Casos de prueba (P1 y P2)

- **P1, dentro de la pantalla.** B = 60 − 20 + 16 = 56 (38 en hexadecimal) y C = 100 − 50 + 8 = 58 (3A). DE y HL no cambian.
- **P2, fuera de la pantalla.** B = 2 − 20 + 16 = −2, que en 8 bits es 254 (FE), y C = 3 − 50 + 8 = −39, que es 217 (D9). Quedan fuera de la pantalla y no hizo falta comprobarlo. DE y HL no cambian.

ⓘ Fíjate en

- Cada resultado sobreescribe a su cámara: cuando has restado, ya no la necesitas.
- No hay que comprobar si se ve: fuera de la pantalla, la cuenta da la vuelta a valores que la consola no dibuja (P2).

⚠ Errores típicos

Nos pasa a todos:

- Restar la cámara Y a la X (y al revés).
- Usar `HL` como intermedio: el contrato dice que no se modifica.

Ejercicio 9: sys_prerender_one: los 2 sprites de una entidad (completar)

ⓘ Pasos de la solución

1. Sprite 1, `Y`: `ld a, b` y `ld [hl+], a`, igual que el sprite 0.
2. `X`: `ld a, c`, `bit ATTR_XFLIP_BIT, e`, `jr nz, .right` y `add SPRITE_WIDTH`. En `.right`, `ld [hl+], a`. Sin volteo va a la derecha (`C + 8`); con volteo, a la izquierda (`C`).
3. `TILE`: `ld a, d`, `add TILES_PER_SPRITE` y `ld [hl+], a`.
4. `ATTR`: `ld a, e` y `ld [hl+], a`.
5. `ld a, l` y `ld [shadow_cursor], a`: `HL` ya está 8 bytes más allá. `ret`.

sys_prerender_one

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Prepares ONE entity to be drawn. Its image is 16 x 16 pixels, drawn with 2
;; sprites of 8 x 16 (each one shows two tiles, one above the other):
;;      sprite 0: the left half, its tile is entity.TILE
;;      sprite 1: the right half, 8 pixels to the right, its tile is entity.TILE + 2
;; If the image is mirrored (ATTR_XFLIP), each half is mirrored and they swap sides.
;; ⚠️ Precondition: the entity has CMP_SPRITE.
;; 📥 INPUT: DE: entity
;; ❌ May modify A, BC, DE, HL
sys_prerender_one::
   ld   h, d                    ;; \ HL = &entity.Y
   ld   a, e                    ;; |
   add  E_Y                     ;; |
   ld   l, a                    ;; /

   ld   a, [hl+]                ;; \ D = entity.Y (HL -> X)
   ld   d, a                    ;; /
   ld   e, [hl]                 ;; E = entity.X
   call world_to_oam            ;; B = OAM Y, C = OAM X of the left half

   ld   a, b                    ;; \ Does its bottom (OAM Y) reach the HUD, or
   cp   HUD_WINDOW_Y + 1        ;; | is it below the screen (the Y wrapped around)?
   ret  nc                      ;; / Yes: not drawn

   inc  l                       ;; \ D = entity.TILE
   ld   d, [hl]                 ;; /
   inc  l                       ;; \ E = entity.ATTR
   ld   e, [hl]                 ;; /

   ld   h, HIGH(shadow_oam)     ;; \ HL = where to write: the shadow OAM + the cursor
   ld   a, [shadow_cursor]      ;; |
   ld   l, a                    ;; /

   ld   a, b                    ;; \ Sprite 0: the left half
   ld   [hl+], a                ;; | Y
   ld   a, c                    ;; | X: normally on the left; mirrored, on the right
   bit  ATTR_XFLIP_BIT, e       ;; |
   jr   z, .left                ;; |
   add  SPRITE_WIDTH            ;; |
.left
   ld   [hl+], a                ;; |
   ld   a, d                    ;; | TILE
   ld   [hl+], a                ;; |
   ld   a, e                    ;; | ATTR
   ld   [hl+], a                ;; /

   ld   a, b                    ;; \ Sprite 1: the right half
   ld   [hl+], a                ;; | Y
   ld   a, c                    ;; | X: normally on the right; mirrored, on the left
   bit  ATTR_XFLIP_BIT, e       ;; |
   jr   nz, .right              ;; |
   add  SPRITE_WIDTH            ;; |
.right
   ld   [hl+], a                ;; |
   ld   a, d                    ;; | TILE + TILES_PER_SPRITE (the right half)
   add  TILES_PER_SPRITE        ;; |
```

```
57      ld   [hl+], a                ;; |
58      ld   a, e                    ;; | ATTR
59      ld   [hl+], a                ;; /
60
61      ld   a, l                    ;; \ shadow_cursor = the next free sprite
62      ld   [shadow_cursor], a      ;; /
63
64      ret
```

Casos de prueba (P1 y P2)

- **P1, sin voltear.** La posición en la OAM es (Y = 56, X = 58). Sprite 0 (la mitad izquierda): 38 3A 50 00. Sprite 1 (la derecha): 38 42 52 00, con X = 58 + 8 y TILE = 80 + 2. shadow_cursor avanza 8: de 40 a 48 (30 en hexadecimal).
- **P2, volteada.** Con el volteo, las mitades intercambian la X. Sprite 0: 38 42 50 20 (a la derecha). Sprite 1: 38 3A 52 20 (a la izquierda). Los dos llevan ATTR = 20. shadow_cursor = 48 (30).

Fíjate en

- La condición del sprite 1 es la contraria a la del sprite 0 (`jr nz` frente a `jr z`): así las dos mitades intercambian su `X` con el volteo.

Errores típicos

Nos pasa a todos:

- Copiar la condición del sprite 0: con volteo, las dos mitades quedan en el mismo sitio.
- Olvidar guardar `shadow_cursor` : la siguiente entidad escribe encima.
- Sumar 1 al tile en lugar de 2: un sprite de 8 × 16 ya muestra 2 tiles.

Ejercicio 10: sys_animation_one: la imagen de cada entidad (corregir el código)

Los errores

Tipo	Dónde	Corrección
E	`jr nz, .tick` (tras `and CMP_PHYSICS`)	Salta a `.tick` cuando la entidad SÍ tiene `PHYSICS`: al revés. Sin `PHYSICS` (`Z`) se anima siempre: `jr z, .tick`.
F	falta `rrca`	El bit 3, aislado, vale 0 u 8, pero el segundo paso empieza 4 tiles después (`TILES_PER_IMAGE`): `rrca` lo divide entre 2.
F	falta `add a`	Cada entrada de `DirImage` ocupa 2 bytes: el índice es `DIR × 2` (`add a` antes de `ld c, a`).
N	`ld hl, DirImages`	La tabla se llama `DirImage`: `DirImages` no existe (lo dice el ensamblador).
E	`res ATTR_YFLIP_BIT, a`	El volteo de la dirección es el horizontal: hay que borrar `ATTR_XFLIP_BIT` antes de poner el de la dirección. Si no, una entidad que mira a la derecha después de mirar a la izquierda seguiría volteada.

ⓘ Pasos de la solución

1. Escribe la fórmula, `TILE` = `SHEET` + fila + paso, y las dos reglas: con `PHYSICS`, solo se anima si se mueve; sin `PHYSICS`, siempre.
2. Primer bloque: `and CMP_PHYSICS` da `Z` si no tiene `PHYSICS`, y entonces se anima siempre: `jr z, .tick`.
3. El paso: `TICK AND ANIM_STEP_MASK` vale 0 u 8; el segundo paso son 4 tiles: falta `rrca`.
4. El índice de la tabla: cada entrada son 2 bytes: falta `add a` (`DIR × 2`).
5. Ensambla: `DirImages` no existe, la tabla es `DirImage`.
6. El volteo: se borra `ATTR_XFLIP_BIT` (el horizontal) y se pone el de la dirección.
7. Comprueba con P1: `DIR_LEFT`, `TICK` = 7 y camina. `TICK` = 8, paso = `8 / 2 = 4`, fila 16 y volteo: `TILE = 80 + 16 + 4 = 100`.

sys_animation_one (corregida)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Animation system, for ONE entity. The image it shows is:
;;      TILE = SHEET + the row of its direction + the image of its step
;; and ATTR gets the flip of its direction. TICK counts the frames it has been walking;
;; its step changes every 8 frames, so it alternates between its 2 images.
;; An entity with PHYSICS only animates while it moves (VY or VX not 0): when it
;; stands still, it shows the first image. The others (a portal) always animate.
;; ⚠️ Precondition: the entity has CMP_ANIMATION.
;; 📥 DE = entity
;; ❌ Modifies A, BC, HL
sys_animation_one::
   ld   a, [de]                 ;; \ A = entity.CMP
   and  CMP_PHYSICS             ;; | Does it have PHYSICS?
   jr   z, .tick                ;; / No: it always animates

   ld   h, d                    ;; \ A = VY | VX: it is 0 only if it is standing still
   ld   a, e                    ;; | (HL = &entity.VY: same page as the entity,
   add  E_VY                    ;; | the offset added to the low byte)
   ld   l, a                    ;; |
   ld   a, [hl+]                ;; |
   or   [hl]                    ;; /
   jr   nz, .tick               ;; Moving: it animates

   ld   h, d                    ;; \ Standing still: TICK = 0, the first image
   ld   a, e                    ;; | (HL = &entity.TICK)
   add  E_TICK                  ;; |
   ld   l, a                    ;; |
   ld   [hl], 0                 ;; |
   jr   .image                  ;; /

.tick
   ld   h, d                    ;; \ One more frame walking:
   ld   a, e                    ;; | entity.TICK++
   add  E_TICK                  ;; | (HL = &entity.TICK)
   ld   l, a                    ;; |
   inc  [hl]                    ;; /

.image
   ld   a, [hl]                 ;; \ A = TICK
   and  ANIM_STEP_MASK          ;; | Bit 3 of TICK: 0 for 8 frames, 8 for the next 8...
   rrca                         ;; | / 2: 0 or 4 (TILES_PER_IMAGE): its step
   ld   b, a                    ;; / B = the tile offset of its step (0 or 4)

   ld   h, d                    ;; \ A = entity.DIR
   ld   a, e                    ;; |
   add  E_DIR                   ;; |
   ld   l, a                    ;; |
   ld   a, [hl]                 ;; /
   add  a                       ;; \ BC = DIR * 2: the offset in the table
   ld   c, a                    ;; | (B moves to A for a moment)
   ld   a, b                    ;; | A = the offset of the step
   ld   b, 0                    ;; /
   ld   hl, DirImage            ;; \ HL = &DirImage[DIR]
   add  hl, bc                  ;; /

   add  [hl]                    ;; \ A = step offset + tile of the row of its direction
```

```
    inc  hl                     ;; | (HL -> the flip)
    ld   b, [hl]                ;; / B = the flip of its direction
    ld   c, a                   ;; C = that tile (A is needed to build the addresses)

    ld   h, d                   ;; \ HL = &entity.SHEET
    ld   a, e                   ;; |
    add  E_SHEET                ;; |
    ld   l, a                   ;; /
    ld   a, c                   ;; \ C = the tile of the image:
    add  [hl]                   ;; | that tile + entity.SHEET
    ld   c, a                   ;; /

    ld   h, d                   ;; \ HL = &entity.TILE
    ld   a, e                   ;; |
    add  E_TILE                 ;; |
    ld   l, a                   ;; /
    ld   [hl], c                ;; \ entity.TILE = C
    inc  l                      ;; / (HL -> ATTR: the next field)

    ld   a, [hl]                ;; \ entity.ATTR = its ATTR with the flip of
    res  ATTR_XFLIP_BIT, a      ;; | its direction (the other bits stay)
    or   b                      ;; |
    ld   [hl], a                ;; /

    ret
```

Casos de prueba (P1 y P2)

- **P1, camina a la izquierda.** Camina (VX = −1), así que TICK pasa de 7 a 8. El bit 3 de TICK vale 1: el paso son 4 tiles. DIR_LEFT: fila 16 y volteo en X. TILE = 80 + 16 + 4 = 100 (64 en hexadecimal) y ATTR = 20.
- **P2, quieta.** Está quieta: TICK vuelve a 0 (la primera imagen). DIR_UP: fila 8 y sin volteo. TILE = 80 + 8 + 0 = 88 (58 en hexadecimal). ATTR pasa de 30 a 10: la paleta se conserva y el volteo (bit 5) se borra, porque mira hacia arriba.

Fíjate en

- Con `jr nz`, el portal (sin `PHYSICS`) no se animaría nunca y el héroe se animaría aunque estuviera quieto.
- `ATTR` puede traer otros bits (la paleta del héroe herido): por eso solo se borra el bit del volteo.

Errores típicos

Nos pasa a todos:

- Arreglar solo lo que encuentra el ensamblador: los errores de lógica no impiden ensamblar.
- Dividir `TICK` entre 2 antes de aplicar `ANIM_STEP_MASK`: el paso cambiaría cada 16 frames.